

Analisis Pergerakan Pengunjung dan Hubungan Antar Lokasi di Area Wisata

Anggota Kelompok:

1. Araf Rahman Maulana - 122450002 (Ketua Kelompok)
2. M. Deriansyah Okutra - 122450101 (Anggota Kelompok)
3. Daffa Ahmad Naufal - 122450137 (Anggota Kelompok)

Deskripsi pembagian pekerjaan:

1. Araf Rahman Maulana (Ketua Kelompok)
 - Identifikasi dataset
 - Ide cerita
 - Kode Preprocessing
 - Analisis
 - PPT
2. M. Deriansyah Okutra (Anggota Kelompok)
 - Kode Preprocessing
 - Kode Visual
 - Pertanyaan
 - Analisis
 - PPT
3. Daffa Ahmad Naufal (Anggota Kelompok)
 - Kode Visual
 - Analisis
 - PPT
 - Video
 - Kesimpulan

1. Pre-Implementation

1.1. Dataset dan Deskripsi:

Dataset yang kami pakai adalah dataset terkait dengan seberapa sering tempat yang ada di dalam data dikunjungi, di dalam dataset terdapat kolom from, yang dimana kolom tersebut berarti titik awalnya, kemudian ada kolom to, yang berarti titik tujuannya, lalu ada kolom cost pada dataset tidak dijelaskan satuan jaraknya itu apa, disini kami menyimpulkan dengan satuan jarak yang biasanya dipakai dalam dataset, yaitu satuannya meter, bukan km, karena pada dataset angkanya sangat banyak, jadi untuk satuannya km itu kurang cocok, kemudian ada kolom profit, yang sudah kami analisis kolom profit ini bukan keuntungan, tapi berapa banyak pengunjung, kemudian ada

kolom category, disini artinya sama aja seperti kolom to, yang berarti tujuannya, disini setiap titik dan tujuan berbeda beda jaraknya.

1.2. Pertanyaan Analisis Visualisasi:

- Apakah semua lokasi ini berhubungan ?
- Destinasi mana yang menjadi destinasi Terpopuler dan Tersepi ?
- Apakah ada hubungan antara jarak dan jumlah pengunjung ?
- Berapa jumlah pengunjung dari setiap Destinasi yang ada ?
- Berapa jumlah pengunjung ke masing-masing destinasinya?
- Bagaimana pola keseluruhan perpindahan pengunjung antara lokasi awal dan lokasi tujuan ?

1.3. Ide Cerita:

Laporan ini berfokus pada analisis pola pergerakan pengunjung antar destinasi untuk memahami keterhubungan lokasi, tingkat popularitas, dan faktor-faktor yang memengaruhi pilihan perjalanan. Berdasarkan data lokasi awal dan tujuan, analisis dilakukan untuk menentukan destinasi terpopuler dan tersepi, mengidentifikasi pola perpindahan antar lokasi, serta mengeksplorasi hubungan antara jarak dan jumlah pengunjung. Hasil analisis ini akan membantu mengungkap tren perjalanan pengunjung, memberikan wawasan mengenai distribusi jumlah pengunjung per destinasi, dan menyusun rekomendasi strategis untuk meningkatkan pengelolaan destinasi. Visualisasi seperti heatmap, bar chart, dan scatter plot digunakan untuk mendukung pemahaman dan pengambilan keputusan.

2. Implementation

2.1. Preprocessing Dataset

Preprocessing data adalah proses awal dalam pengolahan data untuk memastikan data bersih, terstruktur, dan siap digunakan dalam analisis atau visualisasi, pada data yang kami gunakan ini, kami melakukan beberapa langkah Preprocessing data sebagai berikut:

1. Mengubah atribut kolom pada from dan to (mengubah baris numerik menjadi baris kategorik)
2. Menghapus kolom "Category"
3. Mengganti nama pada kolom "from", "to", "cost", dan "profit"
4. Mengganti atau menghapus titik dari atribut jarak
5. Menghapus kolom jarak (meter) Normalisasi dengan scalling pada atribut jarak
6. Mengelompokkan lokasi spesifik ke dalam kategori yang lebih umum
7. Menghapus data duplikat pada kolom lokasi awal gabungan dan lokasi tujuan gabungan
8. Memfilter dataset untuk menyimpan baris baris dalam kategori tertentu
9. Memfilter dataset apakah masih ada yang kosong

Berikut adalah kode yang kami gunakan untuk menerapkan langkah-langkah preprocessing di atas:

2.1.1 Replace

Fungsi replace digunakan untuk mengganti nilai-nilai tertentu dalam kolom Data Frame dengan nilai baru berdasarkan aturan yang ditentukan. Dalam kode ini, replace mengganti angka-angka kategori pada kolom 'from' dan 'to' dengan nama-nama kategori seperti "Palace", "Zoo", dan lainnya, sesuai dengan mapping yang didefinisikan dalam dictionary category mapping. Tujuannya adalah untuk mempermudah interpretasi data, sehingga angka kategori menjadi lebih bermakna dan mudah dibaca.

Input :

from	to	cost	profit	category
1	2	740.592.073.561.656	1478	Zoo
1	3	719.161.300.154.117	1243	Entertainment
1	4	470.482.001.718.146	1042	Museum
1	5	465.753.807.495.913	1750	Palace
1	6	42.511.484.871.629	1581	Museum

Output :

from	to	cost	profit	category
Palace_1	Zoo_1	740.592.073.561.656	1478	Zoo
Palace_1	Entertainment_1	719.161.300.154.117	1243	Entertainment
Palace_1	Museum_1	470.482.001.718.146	1042	Museum
Palace_1	Palace_2	465.753.807.495.913	1750	Palace
Palace_1	Museum_2	42.511.484.871.629	1581	Museum

Code Program

```
import pandas as pd

# Membuat mapping kategori
```

```

category_mapping = {
    7: "Palace", 5: "Palace", 1: "Palace",
    10: "Zoo", 2: "Zoo",
    15: "Entertainment", 3: "Entertainment",
    28: "Museum", 22: "Museum", 21: "Museum", 19: "Museum",
    16: "Museum", 14: "Museum", 13: "Museum", 11: "Museum",
    9: "Museum", 6: "Museum", 4: "Museum",
    8: "Structure",
    26: "History", 25: "History", 23: "History", 20: "History",
    18: "History", 17: "History",
    29: "Park", 24: "Park",
    27: "Cultural"
}

# Mengganti nilai pada kolom from dan to berdasarkan mapping
df['from'] = df['from'].replace(category_mapping)
df['to'] = df['to'].replace(category_mapping)

# Menampilkan dataframe hasil
df

```

2.1.2 Remove

Removed digunakan untuk menghapus baris atau kolom dari DataFrame. Dalam kode ini, `df.drop(columns=["category"])` menghapus kolom bernama "category" dari DataFrame.

Output :

	from	to	cost	profit
	Palace_1	Zoo_1	740.592.073.561.656	1478
	Palace_1	Entertainment_1	719.161.300.154.117	1243
	Palace_1	Museum_1	470.482.001.718.146	1042
	Palace_1	Palace_2	465.753.807.495.913	1750
	Palace_1	Museum_2	42.511.484.871.629	1581

Code Program

```

df=df.drop(columns=["category"])
df

```

2.1.3 Rename

Rename digunakan untuk mengubah nama kolom dalam DataFrame. Pada kode tersebut, `df.rename(columns={..}, inplace=True)` mengganti nama kolom-kolom tertentu di DataFrame `df`. Kolom "from" diganti menjadi "Lokasi_Awal", kolom "to" menjadi "Lokasi_Tujuan", kolom "cost" menjadi "Jarak (meter)", dan kolom "profit" menjadi "Jumlah_Pengunjung".

Output :

Lokasi_Awal	Lokasi_Tujuan	Jarak (meter)	Jumlah_Pengunjung
Palace_1	Zoo_1	740.592.073.561.656	1478
Palace_1	Entertainment_1	719.161.300.154.117	1243
Palace_1	Museum_1	470.482.001.718.146	1042
Palace_1	Palace_2	465.753.807.495.913	1750
Palace_1	Museum_2	42.511.484.871.629	1581

Code Program

```
# Rename columns in place
df.rename(columns={
    "from": "Lokasi_Awal",
    "to": "Lokasi_Tujuan",
    "cost": "Jarak (meter)",
    "profit": "Jumlah_Pengunjung"
}, inplace=True)
df
```

2.1.4 Replace

Replace digunakan untuk mengganti nilai-nilai tertentu dalam kolom sama seperti sebelumnya namun disini replacenya lebih ke menghapus titik pada atribut `df['Jarak (meter)']`.

Output :

Lokasi_Awal	Lokasi_Tujuan	Jarak (meter)	Jumlah_Pengunjung
Palace_1	Zoo_1	740592073561656	1478
Palace_1	Entertainment_1	719161300154117	1243
Palace_1	Museum_1	470482001718146	1042
Palace_1	Palace_2	465753807495913	1750
Palace_1	Museum_2	42511484871629	1581

Code Program

```
# Pastikan kolom Jarak (meter) berupa string sebelum menghapus titik
df['Jarak (meter)'] = df['Jarak (meter)'].astype(str).str.replace('.', '', regex=False)

# Jika diperlukan, ubah kembali menjadi angka
df['Jarak (meter)'] = pd.to_numeric(df['Jarak (meter)'], errors='coerce')

df
```

2.1.5 Remove dan Normalisasi

Remove digunakan untuk menghapus sesuatu dari sebuah data, dan Normalisasi digunakan untuk mengubah angka besar jadi lebih kecil dan mudah dikelola. pada kode kali ini kami menghapus atribut jarak (meter) dan menormalisasi data pada atribut jarak.

Output :

Lokasi_Awal	Lokasi_Tujuan	Jumlah_Pengunjung	Jarak
Palace_1	Zoo_1	1478	740.59
Palace_1	Entertainment_1	1243	719.16
Palace_1	Museum_1	1042	470.48
Palace_1	Palace_2	1750	465.75
Palace_1	Museum_2	1581	42.51

Code Format

```
# Pastikan kolom Jarak (meter) numerik
data = df['Jarak (meter)'].values

scaling_factor = 10**12 # normalisasi jarak dari yang sangat besar jadi ratusan agar mempercepat komputasi

# Lakukan normalisasi dan batasi hingga dua angka di belakang koma
df['Jarak'] = (data / scaling_factor).round(2)
df
```

2.1.6 Apply

Apply digunakan untuk menerapkan fungsi ke setiap elemen dalam sebuah kolom atau baris pada DataFrame, Pada kode ini apply digunakan pada kolom Lokasi_Tujuan dan Lokasi_Awal untuk mengganti lokasi yang lebih spesifik menjadi kategori yang lebih umum. Fungsi yang diterapkan adalah fungsi lambda yang memeriksa apakah lokasi ada di dalam daftar tertentu.

Output :

Lokasi_Awal	Lokasi_Tujuan	Jumlah_Pengunjung	Jarak	Lokasi_Tujuan_Gabungan	Lokasi_Awal_Gabungan
Palace_1	Zoo_1	1478	740.59	Zoo	Palace
Palace_1	Entertainment_1	1243	719.16	Entertainment	Palace
Palace_1	Museum_1	1042	470.48	Museum	Palace
Palace_1	Palace_2	1750	465.75	Palace	Palace
Palace_1	Museum_2	1581	42.51	Museum	Palace

Code Program

```
# Menentukan kategori yang ingin digabungkan (sesuai dengan data yang ada)
museum_locations = ['Museum_1', 'Museum_2', 'Museum_3', 'Museum_4', 'Museum_5',
                    'Museum_6', 'Museum_7', 'Museum_8', 'Museum_9', 'Museum_10', 'Museum_11']
park_locations = ['Park_1', 'Park_2']
zoo_locations = ['Zoo_1', 'Zoo_2']
palace_locations = ['Palace_1', 'Palace_2', 'Palace_3']
entertainment_locations = ['Entertainment_1', 'Entertainment_2']
structure_locations = ['Structure']
history_locations = ['History_1', 'History_2', 'History_3', 'History_4', 'History_5', 'History_6']
cultural_locations = ['Cultural']

# Membuat kolom baru untuk kategori yang lebih umum (Museum, Park, Zoo, Palace, Entertainment, Structure, History, Cultural)
df['Lokasi_Tujuan_Gabungan'] = df['Lokasi_Tujuan'].apply(lambda x: 'Museum' if x in museum_locations else
                                                         ('Park' if x in park_locations else
                                                          ('Zoo' if x in zoo_locations else
                                                           ('Palace' if x in palace_locations else
                                                            ('Entertainment' if x in entertainment_locations else
                                                             ('Structure' if x in structure_locations else
                                                              ('History' if x in history_locations else
                                                               ('Cultural' if x in cultural_locations else x))))))))))
```

```
# Membuat kolom baru untuk kategori yang lebih umum (Museum, Park, Zoo, Palace, Entertainment, Structure, History, Cultural)
df['Lokasi_Awal_Gabungan'] = df['Lokasi_Awal'].apply(lambda x: 'Museum' if x in museum_locations else
('Park' if x in park_locations else
('Zoo' if x in zoo_locations else
('Palace' if x in palace_locations else
('Entertainment' if x in entertainment_locations else
('Structure' if x in structure_locations else
('History' if x in history_locations else
('Cultural' if x in cultural_locations else x))))))
df
```

2.1.7 Remove Duplicate

Remove duplicate adalah proses menghapus baris atau data yang duplikat (berulang) dalam suatu DataFrame, Pada Kode ini menghapus baris yang dianggap redundan dengan cara memfilter baris di mana lokasi awal (Lokasi_Awal_Gabungan) dan lokasi tujuan (Lokasi_Tujuan_Gabungan) memiliki nilai yang sama.

Code Program

```
# Hapus baris di mana Lokasi_Awal_Gabungan sama dengan Lokasi_Tujuan_Gabungan
df_unique1 = df[df['Lokasi_Awal_Gabungan'] != df['Lokasi_Tujuan_Gabungan']]
df_unique1.head(5)
```

2.1.8 Filtering

Filtering adalah proses menyaring data dalam Data Frame berdasarkan kondisi tertentu, pada kode ini filtering digunakan untuk membuat subset data berdasarkan kategori pada kolom Lokasi_Tujuan_Gabungan seperti "Museum," "History," "Zoo," dan lainnya. Setiap subset menyimpan hanya baris-baris dengan kategori tertentu, sehingga mempermudah analisis spesifik, seperti menghitung jumlah pengunjung ke lokasi tertentu atau memahami pola pergerakan berdasarkan jenis destinasi.

```
df_museum = df[df['Lokasi_Tujuan_Gabungan'] == 'Museum']
df_history = df[df['Lokasi_Tujuan_Gabungan'] == 'History']
df_entertainment = df[df['Lokasi_Tujuan_Gabungan'] == 'Entertainment']
df_zoo = df[df['Lokasi_Tujuan_Gabungan'] == 'Zoo']
df_palace = df[df['Lokasi_Tujuan_Gabungan'] == 'Palace']
df_park = df[df['Lokasi_Tujuan_Gabungan'] == 'Park']
df_structure = df[df['Lokasi_Tujuan_Gabungan'] == 'Structure']
df_cultural = df[df['Lokasi_Tujuan_Gabungan'] == 'Cultural']
```


2.1.9 Filtering

Filtering digunakan untuk menyaring data dalam Data Frame untuk memenuhi kondisi tertentu, pada kode ini filtering berguna untuk melihat apakah ada data kosong pada Data Frame.

Output :

	0
Lokasi_Awal	0
Lokasi_Tujuan	0
Jumlah_Pengunjung	0
Jarak	0
Lokasi_Tujuan_Gabungan	0
Lokasi_Awal_Gabungan	0

Code Program

```
df.isnull().sum()
```

Link data awal dan data setelah preprocessing :

https://drive.google.com/file/d/1xtCHD0jGizoT2DVfdAE2B5KSNNQcLzOq/view?usp=drive_link (Link data awal)

https://drive.google.com/file/d/1GE28wZlyfeJyv4hJQ28KYYUH0nK5SKFr/view?usp=drive_link (Link data setelah preprocessing)

2.2. Visualization

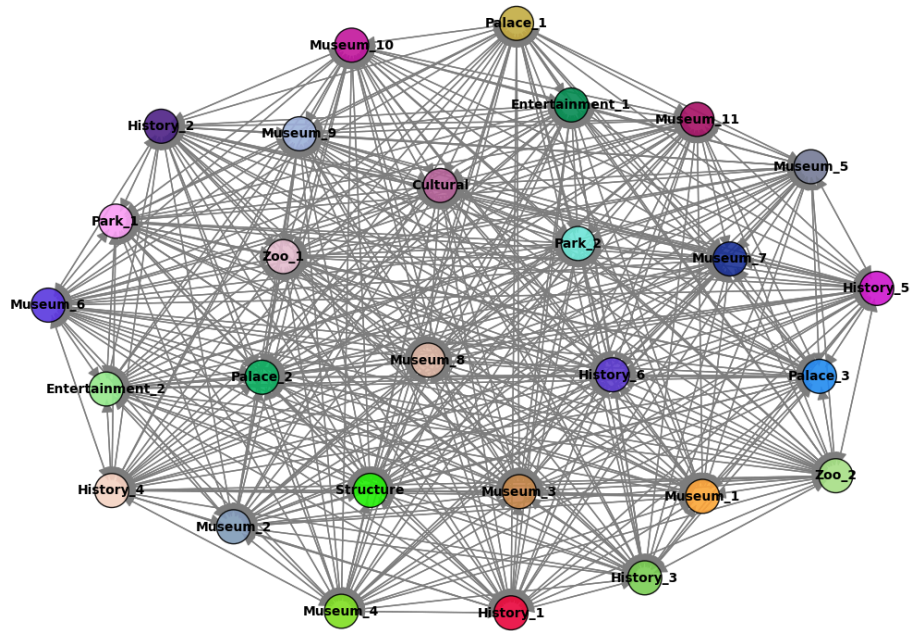
Berdasarkan pertanyaan dari 1.2 kami membuat visualisasi sebagai berikut:

2.2.1. Apakah semua lokasi ini berhubungan ?

Dari pertanyaan tersebut kami membuat visualisasi graph dimana visualisasi ini akan menggambarkan setiap lokasinya sebagai node dan tujuan dari satu lokasi ke lokasi yang lainnya, dan bisa dilihat pada visualisasi ini semua lokasi itu terhubung satu sama lainnya.

Visualisasi Graph

Diagram Jaringan: Lokasi Awal ke Tujuan



Gambar 1

<code>

```
# Membuat Graph berarah
G = nx.DiGraph()

# Menambahkan edges dengan bobot (jumlah pengunjung)
for _, row in df.iterrows():
    G.add_edge(row["Lokasi_Awal"], row["Lokasi_Tujuan"], weight=row["Jarak"])

# Menentukan tata letak untuk visualisasi
pos = nx.spring_layout(G, seed=42) # Tata letak berbasis pegas

# Buat warna acak untuk setiap node
node_colors = {node: f#{random.randint(0, 0xFFFFFF):06x}' for node in G.nodes()}

# Visualisasi Graph
plt.figure(figsize=(14, 10))

# Gambar nodes dengan warna unik
nx.draw_networkx_nodes(
    G, pos, node_size=700, node_color=[node_colors[node] for node in G.nodes()],
    edgecolors="black", alpha=0.9
)

# Gambar edges
nx.draw_networkx_edges(
    G, pos, edge_color="gray", arrowsize=20, arrowstyle="->"
)

# Gambar labels nodes
```

```

labels = {}
for node in G.nodes():
    labels[node] = f"{node}" if node in df["Lokasi_Tujuan"].values else node
nx.draw_networkx_labels(
    G, pos, labels=labels, font_size=10, font_color="black", font_weight="bold"
)

# Menambahkan label pada edges (total pengunjung)
edge_labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels, font_size=8, font_color="darkblue"
)

# Menampilkan diagram
plt.title("Diagram Jaringan: Lokasi Awal ke Tujuan dengan Jarak setiap tujuan", fontsize=16)
plt.axis("off")
plt.show()

```

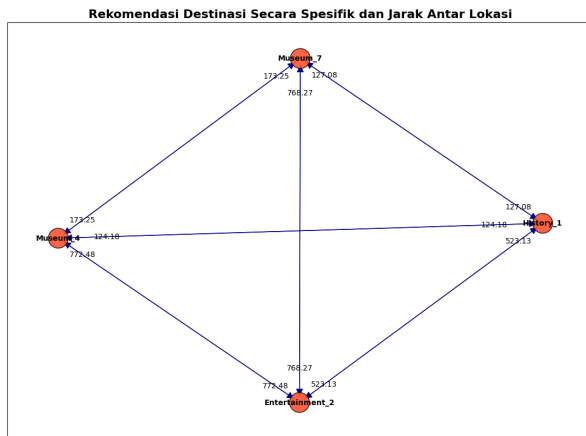
Analisis:

Gambar tersebut merupakan diagram jaringan yang menunjukkan hubungan antar lokasi, dimana setiap node menggambarkan suatu tempat, seperti museum, istana, taman, kebun binatang, dan tempat hiburan lainnya. Node diberi label berdasarkan jenis lokasi, misalnya Museum_1, Istana_1, dll., dengan warna yang muncul untuk menunjukkan kategori lokasi. Jaringan ini sangat padat, hal ini terlihat dari banyaknya koneksi (edge) yang menghubungkan hampir setiap node dengan node lainnya, sehingga menunjukkan adanya koneksi atau hubungan antar lokasi tersebut. Hal ini dapat mewakili hubungan aksesibilitas, seperti rute perjalanan atau interaksi antara lokasi asal dan tujuan. Jaringan yang padat ini menunjukkan bahwa setiap lokasi terhubung dengan baik sehingga memungkinkan banyak rute dari satu titik ke titik lainnya. Struktur ini juga dapat memberikan wawasan tentang tingkat konektivitas yang sangat tinggi antar lokasi, menjadikan jaringan ini kompleks dan fleksibel untuk berbagai jenis analisis, misalnya rute terpendek, popularitas lokasi, atau efisiensi perjalanan antar lokasi. Secara keseluruhan, diagram ini memberikan gambaran visual tentang hubungan antar lokasi dalam sistem perjalanan atau pariwisata.

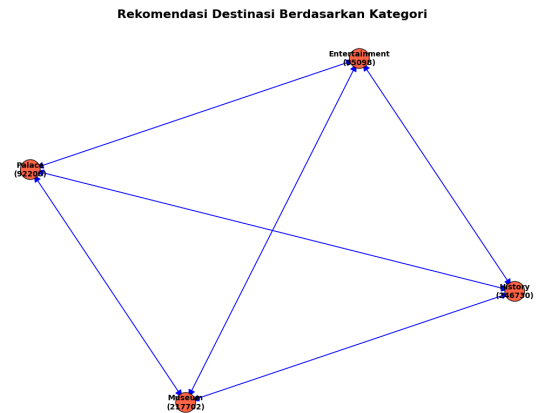
2.2.2 Destinasi mana yang menjadi destinasi Terpopuler dan Tersepi?

Agar lebih mudah memahami pola perjalanan dan popularitas suatu lokasi, kami telah membuat visualisasi grafik. Setiap lokasi direpresentasikan sebagai sebuah simpul (node), dan jarak antara lokasi-lokasi tersebut ditunjukkan oleh garis penghubung (edge) beserta nilai jaraknya. Pada gambar 2.2 dan 2.4 menggunakan preprocessing df_unique1 yg bisa dilihat dari (2.1.7 Remove Duplicate)

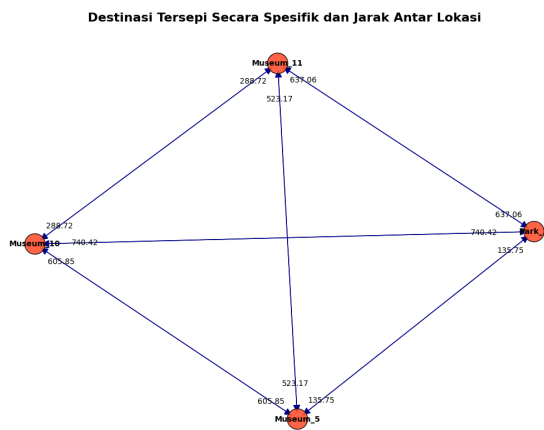
Visualisasi Graph



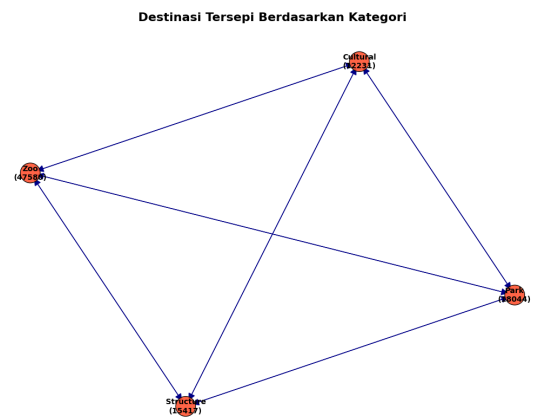
Gambar 2.1



Gambar 2.2



Gambar 2.3



Gambar 2.4

<code gambar 2.1 dan 2.3>

```

# Membuat Graph berarah
G = nx.DiGraph()

# Menghitung total jumlah pengunjung untuk setiap edge
edge_weights = df.groupby(["Lokasi_Awal", "Lokasi_Tujuan"])["Jumlah_Pengunjung"].sum().reset_index()

# Menambahkan edges dengan bobot (total jumlah pengunjung) dan jarak
for _, row in edge_weights.iterrows():
    jarak = df.loc[(df["Lokasi_Awal"] == row["Lokasi_Awal"]) & (df["Lokasi_Tujuan"] == row["Lokasi_Tujuan"]),

```

```

'Jarak'].values[0]
G.add_edge(row["Lokasi_Awal"], row["Lokasi_Tujuan"], weight=row["Jumlah_Pengunjung"], distance=jarak)

# Menghitung total bobot (popularitas) setiap node sebagai tujuan
popularitas = {}
for _, row in edge_weights.iterrows():
    if row["Lokasi_Tujuan"] in popularitas:
        popularitas[row["Lokasi_Tujuan"]] += row["Jumlah_Pengunjung"]
    else:
        popularitas[row["Lokasi_Tujuan"]] = row["Jumlah_Pengunjung"]

# Mengambil 4 destinasi Tersepi
rekendasi = sorted(popularitas.items(), key=lambda x: x[1], reverse=False)[:4]
rekendasi_nodes = [item[0] for item in rekendasi]

# Membuat subgraph hanya dengan 4 destinasi Tersepi dan edges antar mereka
subgraph = G.subgraph(rekendasi_nodes).copy()

# Menentukan tata letak untuk visualisasi
pos = nx.spring_layout(subgraph, seed=42)

# Buat warna acak untuk setiap node
node_colors = {node: f'#{random.randint(0, 0xFFFFFF):06x}' for node in subgraph.nodes()}

# Visualisasi Graph
plt.figure(figsize=(14, 10))

# Gambar nodes
nx.draw_networkx_nodes(
    subgraph, pos, node_size=700,
    node_color=[node_colors[node] if node not in rekendasi_nodes else "#FF5733" for node in subgraph.nodes()],
    edgecolors="black", alpha=0.9
)

# Gambar edges
nx.draw_networkx_edges(subgraph, pos, edge_color="darkblue", arrowsize=20, arrowstyle="->")

# Gambar node labels
node_labels = {node: node for node in subgraph.nodes()}
nx.draw_networkx_labels(subgraph, pos, labels=node_labels, font_size=10, font_color="black", font_weight="bold")

# Menambahkan label jarak pada edges dengan offset
distance_labels = nx.get_edge_attributes(subgraph, "distance")
for (u, v), d in distance_labels.items():
    x_pos = pos[u][0] + 0.1 * (pos[v][0] - pos[u][0]) # Adjust offset as needed
    y_pos = pos[u][1] + 0.1 * (pos[v][1] - pos[u][1])
    plt.text(x_pos, y_pos, f'{d:.2f}', fontsize=10, color="black", ha="center", va="center")

# Judul dan pengaturan
plt.title("Destinasi Tersepi Secara Spesifik dan Jarak Antar Lokasi", fontweight="bold", fontsize=16)
plt.axis("off")
plt.show()

#Note : kode pada gambar 2.1 dan 2.3 itu secara menyeluruh sama cuman pembedanya ada pada bagian : (reverse=True) untuk
gambar 2.1 dan (reverse=False) untuk 2.3

```

<code gambar 2.2 dan 2.4>

```

# Membuat Graph berarah
G = nx.DiGraph()

```

```

# Menambahkan edges dengan bobot (jumlah pengunjung)
for _, row in df.iterrows():
    G.add_edge(row["Lokasi_Awal"], row["Lokasi_Tujuan"], weight=row["Jarak"])

# Menghitung total bobot (popularitas) setiap node sebagai tujuan
popularitas = {}
for _, row in df.iterrows():
    if row["Lokasi_Tujuan"] in popularitas:
        popularitas[row["Lokasi_Tujuan"]] += row["Jarak"]
    else:
        popularitas[row["Lokasi_Tujuan"]] = row["Jarak"]

# Mengambil 4 destinasi terpopuler
rekendasi = sorted(popularitas.items(), key=lambda x: x[1], reverse=True)[:4]
rekendasi_nodes = [item[0] for item in rekendasi]

# Menentukan tata letak untuk visualisasi
pos = nx.spring_layout(G, seed=42) # Tata letak berbasis pegas

# Buat warna acak untuk setiap node
node_colors = {node: f'#{random.randint(0, 0xFFFFFF):06x}' for node in G.nodes()}

# Visualisasi Graph
plt.figure(figsize=(14, 10))

# Gambar nodes dengan warna unik (highlight rekomendasi)
nx.draw_networkx_nodes(
    G, pos, node_size=700,
    node_color=[node_colors[node] if node not in rekendasi_nodes else "#FF5733" for node in G.nodes()],
    edgecolors="black", alpha=0.9
)

# Gambar edges
nx.draw_networkx_edges(
    G, pos, edge_color="gray", arrowsize=20, arrowstyle="->"
)

# Gambar labels nodes
labels = {}
for node in G.nodes():
    labels[node] = f'{node}' if node not in rekendasi_nodes else node
nx.draw_networkx_labels(
    G, pos, labels=labels, font_size=10, font_color="black", font_weight="bold"
)

# Menambahkan label pada edges (total pengunjung)
edge_labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels, font_size=8, font_color="darkblue"
)

# Menampilkan diagram
plt.title("Rekomendasi Destinasi Terpopuler", fontsize=16)
plt.axis("off")
plt.show()

#Note : kode pada gambar 2.2 dan 2.4 itu secara menyeluruh sama cuman pembedanya ada pada bagian : (reverse=True)
untuk gambar 2.2 dan (reverse=False) untuk 2.4

```

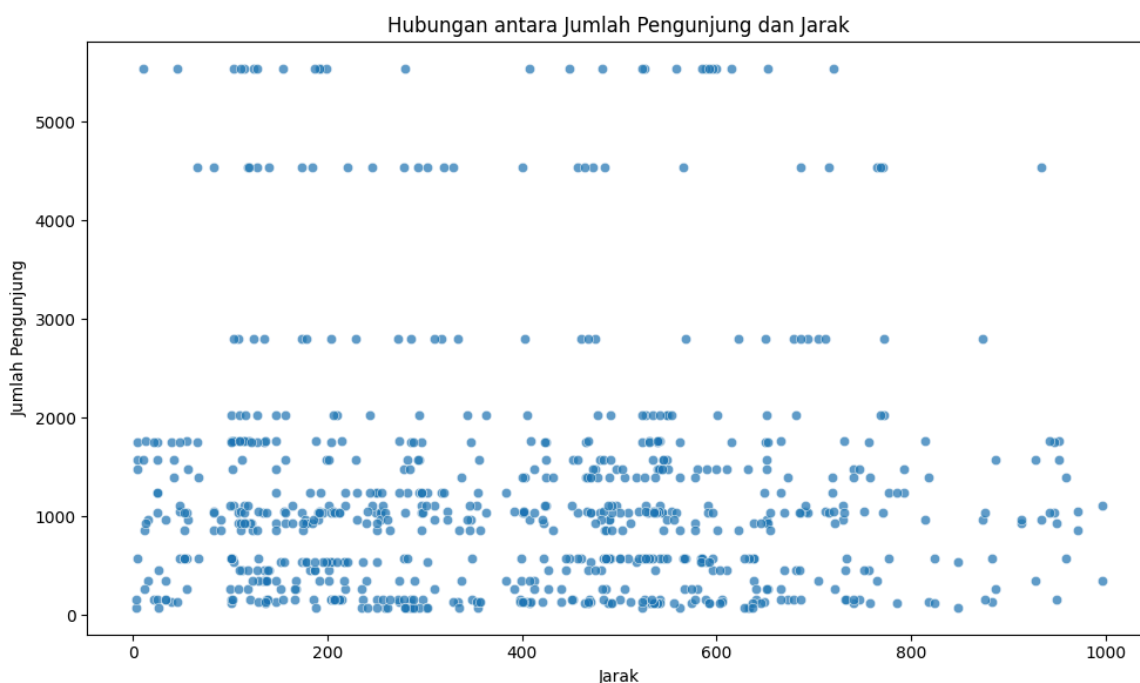
Analisis:

Karena lokasi A memiliki koneksi terbanyak dengan lokasi B, C, D, dan E, dan karena rute terpendek antara A dan C (jarak 3), lokasi C dapat menjadi pilihan terbaik untuk memulai perjalanan. Ini menunjukkan bahwa lokasi A adalah destinasi paling populer berdasarkan visualisasi graph. Secara keseluruhan, lokasi A berfungsi sebagai pusat jaringan ini, baik dari segi popularitas maupun jumlah konektivitas yang dimilikinya.

2.2.3 Apakah ada hubungan antara jarak dan jumlah pengunjung?

Untuk memahami faktor-faktor yang mempengaruhi minat wisatawan dalam mengunjungi berbagai destinasi, kami telah menganalisis data jumlah pengunjung dan jarak tempuh. Visualisasi ini menggambarkan hubungan antara kedua variabel tersebut. Setiap titik pada grafik mewakili satu destinasi wisata, di mana posisi titik di sumbu X menunjukkan jaraknya dari pusat kota, sedangkan posisi di sumbu Y menunjukkan jumlah pengunjung tahunan.

Visualisasi Scatterplot



Gambar 3

<code>

```
# Pastikan kolom dan data valid
data = df.dropna(subset=['Jarak', 'Jumlah_Pengunjung'])
data['Jarak'] = pd.to_numeric(data['Jarak'], errors='coerce')
data['Jumlah_Pengunjung'] = pd.to_numeric(data['Jumlah_Pengunjung'], errors='coerce')

# Scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Jarak', y='Jumlah_Pengunjung', data=data, alpha=0.7)
plt.title('Hubungan antara Jumlah Pengunjung dan Jarak')
```

```
plt.xlabel('Jarak')
plt.ylabel('Jumlah Pengunjung')
plt.tight_layout()
plt.show()
```

Analisis:

Grafik scatter menunjukkan bahwa tidak ada korelasi yang jelas, baik positif maupun negatif, antara jarak dan jumlah pengunjung. Ini ditunjukkan oleh sebaran titik data yang tidak membentuk pola tertentu. Destinasi jauh tetap ramai, meskipun beberapa destinasi dekat (di bawah 200) menarik banyak orang. Oleh karena itu, variabel lain selain jarak, seperti daya tarik destinasi, aksesibilitas, dan promosi, kemungkinan lebih berpengaruh terhadap jumlah pengunjung.

2.2.4 Berapa jumlah pengunjung dari setiap Destinasi yang ada?

Kami membuat visualisasi yang bertujuan untuk menghitung jumlah pengunjung dari setiap kategori lokasi dengan menggunakan batang yang berbeda warna, diagram batang ini menunjukkan jumlah pengunjung untuk setiap kategori lokasi.

Visualisasi Barplot



Gambar(Lokasi Tujuan) 4.1



Gambar(Lokasi Awal) 4.2

<code>


```

# Mengelompokkan data berdasarkan Lokasi_Tujuan_Gabungan dan menghitung jumlah pengunjung
df_grouped = df.groupby('Lokasi_Tujuan_Gabungan')['Jumlah_Pengunjung'].sum().reset_index()

# Menampilkan Data yang telah dikelompokkan
print(df_grouped)

# Menentukan warna dengan menggunakan seaborn palette untuk visualisasi lebih menarik
sns.set(style="whitegrid")
colors = sns.color_palette("Set3", len(df_grouped)) # Warna yang lebih cerah dan berbeda-beda

# Membuat Barchart dengan efek bayangan pada bar
plt.figure(figsize=(12, 8))

# Menggambar bar dengan warna berbeda untuk setiap kategori
bars = plt.bar(df_grouped['Lokasi_Tujuan_Gabungan'], df_grouped['Jumlah_Pengunjung'], color=colors, edgecolor="black")

# Menambahkan detail ke grafik
plt.title("Jumlah Pengunjung Dari Setiap Tempat", fontsize=18, fontweight='bold', color='darkblue')
plt.xlabel("Lokasi Tujuan", fontsize=12, color='black', fontweight='bold')
plt.ylabel("Jumlah Pengunjung", fontsize=12, color='black', fontweight='bold')

# Menyesuaikan rotasi label pada sumbu X dan menambahkan grid pada sumbu Y
plt.xticks(rotation=45, fontsize=12, ha="right")
plt.grid(axis="y", linestyle="--", alpha=0.7)

# Menambahkan label jumlah pengunjung di atas setiap bar dengan penyesuaian font
for i, bar in enumerate(bars):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 5,
             str(int(bar.get_height())), ha="center", fontsize=12, color="black")

# Menambahkan legenda warna untuk setiap kategori
category_labels = df_grouped['Lokasi_Tujuan_Gabungan']
handles = [plt.Rectangle((0, 0), 1, 1, color=color) for color in colors]
plt.legend(handles, category_labels, title="Kategori Lokasi Tujuan", bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=10)

# Menampilkan Grafik
plt.tight_layout()
plt.show()

```

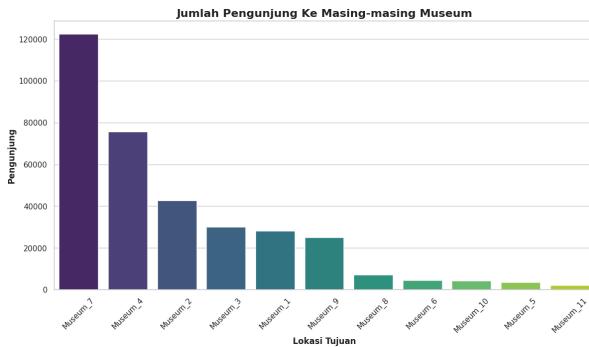
Analisis:

Visualisasi ini menunjukkan jumlah pengunjung di berbagai kategori lokasi; Museum memiliki jumlah tertinggi (345,762) dan Cultural terendah (12,231), masing-masing. Rata-rata jumlah pengunjung adalah 116,538.75, sedangkan median adalah 68,890.5. Ini adalah perbedaan besar antara rata-rata dan median karena kategori seperti Museum dan History memiliki nilai yang sangat tinggi, yang membuat rata-rata lebih tinggi daripada median. Tujuan visualisasi ini adalah untuk mendapatkan pemahaman tentang distribusi pengunjung dan ketimpangan antar kategori.

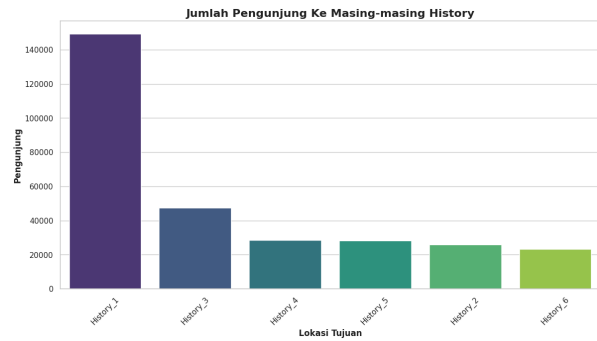
2.2.5 Berapa jumlah pengunjung ke masing-masing destinasinya?(berdasarkan kode 2.1.8)

Kami membuat visualisasi untuk melihat bagaimana pengunjung tersebar ke masing-masing museum. Jumlah pengunjung di sebelas museum ditunjukkan pada diagram batang ini; Museum 7 memiliki jumlah pengunjung tertinggi sekitar 120,000, sementara museum lain, seperti Museum 11, memiliki jumlah pengunjung yang jauh lebih rendah.

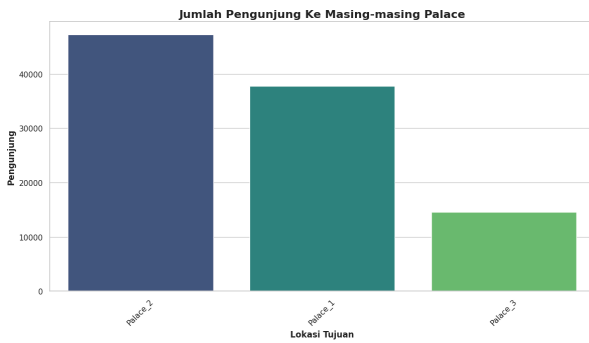
Visualisasi Barplot



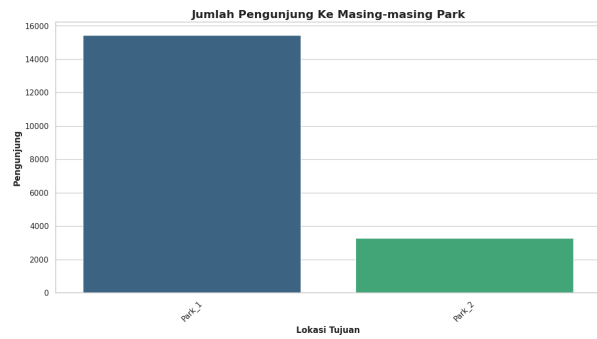
Gambar(Museum) 5.1



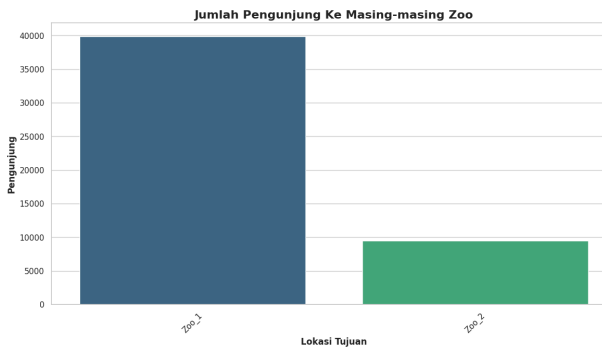
Gambar(History) 5.2



Gambar(Palace) 5.3



Gambar(Park) 5.4



Gambar(Zoo) 5.5



Gambar(Entertainment) 5.6

<code >

```
# Mengelompokkan data berdasarkan lokasi tujuan dan menghitung total pengunjung
total_pengunjung_tujuan = df_destinasi.groupby("Lokasi_Tujuan")["Jumlah_Pengunjung"].sum().reset_index()

# Mengurutkan data berdasarkan total pengunjung
total_pengunjung_tujuan = total_pengunjung_tujuan.sort_values(by="Jumlah_Pengunjung", ascending=False)
print(total_pengunjung_tujuan)
```

```
# Plotting barplot untuk total pengunjung setiap lokasi tujuan
plt.figure(figsize=(14, 7))
sns.barplot(data=total_pengunjung_tujuan, x="Lokasi_Tujuan", y="Jumlah_Pengunjung", palette="viridis")
plt.title("Jumlah Pengunjung Ke Masing-masing Museum", fontsize=16)
plt.xlabel("Lokasi Tujuan", fontsize=12)
plt.ylabel("Pengunjung", fontsize=12)
plt.xticks(rotation=45)
plt.show()
```

#Note: semua kode pada gambar sama yg diubah cuma bagian df destinasi menjadi : df
6.1 Museum, 6.2 Histori, 6.3 palace, 6.4 Park, 6.5 zoo, 6.6 Entertainment

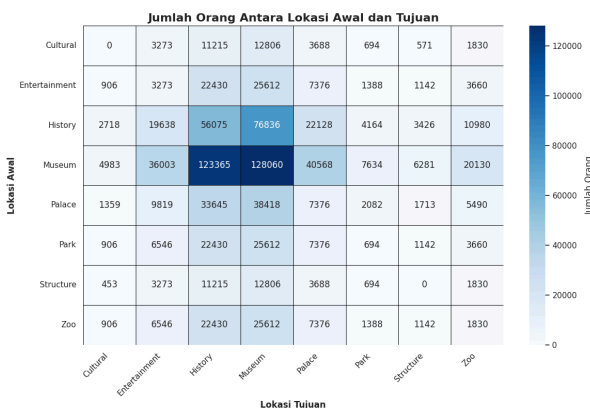
Analisis:

Secara keseluruhan, dari grafik tersebut terlihat adanya ketimpangan jumlah pengunjung antara beberapa lokasi wisata. Lokasi yang memiliki jumlah pengunjung tinggi mungkin memiliki keunggulan yang signifikan, sementara lokasi dengan pengunjung rendah perlu mengidentifikasi faktor yang menyebabkan rendahnya daya tarik dan memperbaikinya untuk meningkatkan kunjungan.

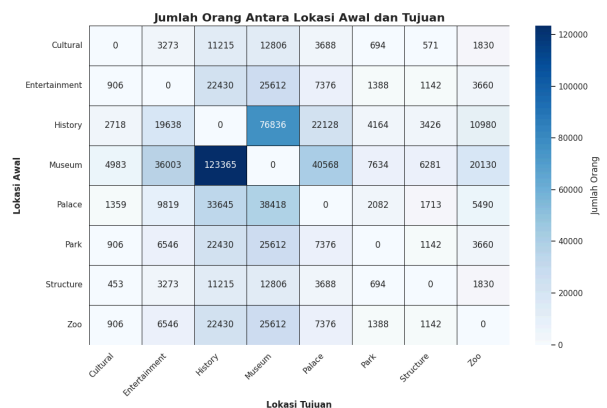
2.2.6 Bagaimana pola keseluruhan perpindahan pengunjung antara lokasi awal dan lokasi tujuan ?

Kami membuat visualisasi dalam bentuk heatmap. Visualisasi ini dirancang untuk menunjukkan jumlah perpindahan pengunjung dari setiap lokasi awal ke lokasi tujuan secara menyeluruh. Pada gambar 6.2 ini menggunakan data prosesing (2.1.7 Remove Duplicate) yg disimpan pada df_unique

Visualisasi Heatmap



Gambar(df) 6.1



Gambar(df_unique1) 6.2

<code>

```
# Menghitung total jumlah orang untuk setiap pasangan lokasi awal dan tujuan
pivot_table = df.pivot_table(
```

```

index="Lokasi_Awal",
columns="Lokasi_Tujuan",
values="Jumlah_Pengunjung",
aggfunc="sum",
fill_value=0
)

# Menampilkan tabel pivot (opsional, untuk verifikasi)
print(pivot_table)

# Membuat heatmap untuk visualisasi
plt.figure(figsize=(12, 8))
sns.heatmap(
    pivot_table,
    annot=True,
    fmt=".0f",
    cmap="Blues",
    linewidths=0.5,
    linecolor="black",
    cbar_kws={"label": "Jumlah Orang"}
)

# Menambahkan judul dan label
plt.title("Jumlah Orang Antara Lokasi Awal dan Tujuan", fontsize=16, fontweight="bold")
plt.xlabel("Lokasi Tujuan", fontsize=12)
plt.ylabel("Lokasi Awal", fontsize=12)
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)

# Menampilkan heatmap
plt.tight_layout()
plt.show()

#Note: kedua kode sama pembeda hanya di df untuk gambar 7.1 dan df_unique untuk gambar 7.2

```

Analisis :

Secara keseluruhan, heatmap ini memberikan wawasan tentang pola kunjungan dan preferensi pengunjung, dengan museum dan lokasi sejarah menjadi pusat gravitasi utama dalam pola perjalanan, sementara lokasi lain berfungsi sebagai pelengkap yang memperkaya pengalaman wisata.

3. Kesimpulan

Kesimpulan yang didapat menunjukkan bahwa visualisasi data adalah alat yang sangat efektif untuk memahami pola dan hubungan dalam kumpulan data yang kompleks. Dalam kasus yang dianalisis, heatmap memberikan wawasan tentang distribusi dan pola pergerakan pengunjung di berbagai lokasi wisata, seperti museum, taman, dan monumen bersejarah yang berbeda. Museum dan situs bersejarah memegang peranan dominan sebagai pusat perhatian wisatawan, dengan intensitas kehadiran yang signifikan sebagai tempat pemberangkatan dan kedatangan. Pada tugas besar ini menyoroti pentingnya memilih visualisasi yang tepat untuk menyampaikan informasi dengan jelas, di mana peta panas digunakan untuk secara efektif menunjukkan korelasi dan kuantitas perpindahan massa antar lokasi. Selain itu, pada analisis ini menyoroti peran visualisasi data dalam mendukung pengambilan keputusan berdasarkan data, seperti mengidentifikasi lokasi

dengan potensi pengunjung tertinggi atau menentukan strategi untuk meningkatkan pengalaman perjalanan. Secara keseluruhan, tugas ini berhasil mengintegrasikan konsep visualisasi data untuk memberikan analisis yang informatif dan konsisten dengan tujuan awal proyek.